

FPGA Implementation of LMS-based FIR Adaptive Filter for Real Time Digital Signal Processing Applications

Carlo Safarian, Tokunbo Ogunfunmi, Walter J. Kozacky
Dept. of Electrical Engineering,
Santa Clara University
Santa Clara, CA 95053, USA
togunfunmi, csafarian, wkozacky@scu.edu

B.K.Mohanty
Department of Electronics and Communication Eng.
Jaypee University of Engineering and Technology
AB-Road, Guna, Madhya Pradesh, INDIA
bk.mohanti@juet.ac.in

Abstract— In this paper, we study existing designs proposed for the FPGA implementation of an LMS adaptive filter. Excess use of multipliers and longer cycle periods are a few of the issues associated with the existing structures. Based on this study, we propose a design for an FPGA implementation of an LMS based adaptive filter using the Xilinx DSP48. The proposed architecture uses one set of multipliers for both filter output and weight-increment term computation. We have simulated the proposed design for a 12-tap adaptive filter in Xilinx system generator and implemented the filter using the Vivado tool set. Implementation results shows that the proposed architecture uses 3 DSP48 units compared to 36 DSP48 units for the existing architecture with the same filter of size 12, and our implementation supports a 75% higher clocking frequency than the existing design. Additionally, the proposed architecture consumes nearly 4.7 times less dynamic power than the existing architecture. Therefore, the proposed architecture is suitable for efficient FPGA realization of an LMS FIR adaptive filter for real-time digital signal processing applications.

Keywords— *FPGA; ASIC; LMS Adaptive Filters;*

I. INTRODUCTION

Adaptive filters are widely used in many areas of digital signal processing applications such as echo cancellation, channel equalizer, noise cancellation and industrial applications [1]. Least mean square (LMS) algorithm is one the most popular adaptive filter algorithms due to its simplicity for implementation and satisfactory convergence behavior. During last decade, several efficient design schemes have been suggested for efficient realization of LMS-based adaptive filter real-time applications [8]-[13]. Most of these design schemes are proposed for implementation of LMS-based adaptive filter in ASIC platform which offers many useful features such as low-power, higher throughput and small area. However, ASIC implementations are expensive and involve longer design cycles. Besides, ASIC designs cannot be reprogrammed and reused for different applications. Although, there are many efficient ASIC designs available in the literature, they have failed to get the attention of industry community. On the other hand, FPGA involves less design

cycle time, reprogrammable and less expensive. With the evolution of high bandwidth and low-power FPGA devices, complex digital circuits can easily be realized in FPGA for various real-time applications. Due to these features, FPGA designs are increasingly popular in the industry community. Currently, many complex digital signal processing algorithms have been implemented in FPGA for consumer electronics, automobile and many others manufacturing sectors.

Few schemes have been proposed and found in the literature for FPGA implementation of LMS-based adaptive filters [2]-[5]. Bai *et al.* have implemented LMS-based adaptive filter for channel equalizer in FPGA using embedded multiplier [2]. They have used transverse filter structure to implement the filter. Yaqin *et al.* [3] have implemented LMS-based adaptive filter in FPGA employing extreme DSP and Xilinx smart IP-core. They have considered the direct form FIR filter structure and assigned one multiplier for each filter tap along with a cascaded adder to sum the products. Shenming *et al.* [4] have proposed parallel structure and scheduling scheme for FPGA implementation of LMS-based adaptive filter used in hearing aid. They have considered normalized LMS (NLMS) where the convergence factor is power normalized. A scheduling scheme is proposed to compute filtering, weight increment vector and the power normalized convergence factor using one set of multipliers. Cai *et al.* [5] have implemented LMS adaptive filter in FPGA using Quartus II.

One of the main issues related to the existing LMS adaptive filter designs is the use of direct-form structure for FIR filter in the filtering section. The direct form structure uses the current cycle weight-vector for computation of current filter output while the transpose-form structure uses filter coefficients of N past clock cycles to compute the current filter output, where N is the filter length of the adaptive filter. Since, N past errors are required for updating the filter weights for N clock cycles, the LMS algorithm with a transpose-form FIR filter is nothing but the delay LMS algorithm (DLMS). The DLMS is similar to LMS algorithm except that it uses past error for updating the filter weight instead on current error.

Therefore, DLMS has degraded error performance compared to the LMS. But, DLMS algorithm offers pipelining and it has shorter iteration period than the LMS-based adaptive filter.

In this paper, we explore some resource sharing techniques such as multiply and accumulation which can be used to reduce the total number of multiplier usage in design. The Xilinx dedicated DSP48 slices have multiplier accumulator (MAC) feature [6]. The implementation of FIR filter can be done using MAC feature of DSP48. Reducing the total number of DSP48 usage in the design offers area and power savings. We have used Vivado tool set and XSG for rapid prototyping the LMS filter. We used DSP48 features in our implementation for area and delay optimization with reduced power consumption. The remainder of this paper is as follows: Section II gives a brief summary of LMS algorithm and proposed architecture. Section III explains FPGA implementation. Section IV shows the simulation results. Section V is conclusion of paper.

II. LMS ADAPTIVE FILTER ALGORITHM AND PROPOSED ARCHITECTURE

A. LMS Adaptive Algorithm

The LMS adaptive filter has a filtering section and one adaptation section. The filtering section uses a finite impulse response (FIR) filter while the adaptation section uses LMS algorithm. In every iteration, the filtering section compute a filter output from which an error value is computed. The adaptive section uses the error value to compute the weight-increment term for updating the weight-vector for the next iteration

The n -th iteration weight-vector $\{\mathbf{w}(n)\}$ of an LMS adaptive filter is updated for the $(n+1)$ -th iteration using the relation:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n) \cdot e(n) \quad (1)$$

where, μ is the convergence factor.

The N -point weight-vector $\mathbf{w}(n)$ is defined as

$$\mathbf{w}(n) = [w_n(0), w_n(1), \dots, w_n(N-1)]^T \quad (2)$$

and the error $e(n)$ is computed as

$$e(n) = d(n) - y(n) \quad (3)$$

where, $d(n)$ is the desired response and $y(n)$ is the an estimate of the desired response which is computed as

$$y(n) = \mathbf{x}(n)^T \mathbf{w}(n) \quad (4)$$

where, the N -point input vector $\mathbf{x}(n)$ is defined as

$$\mathbf{x}(n) = [x(n), x(n-1), \dots, x(n-N+1)]^T \quad (5)$$

B. Proposed Architecture

The LMS adaptive filter of length N can be decomposed into K vector of size L , where $N=KL$. The filter computation of (4) can be expressed as

$$y(n) = \sum_{m=0}^{K-1} y_m(n) \quad (6)$$

$$y_m(n) = \sum_{l=0}^{L-1} w_{mL+l}(n) \cdot x(n - mL - l) \quad (7)$$

The small weight vectors $\mathbf{w}_m(n)$ can be updated by following equation

$$\mathbf{w}_m(n+1) = \mathbf{w}_m(n) + \mu \mathbf{x}_m(n) \cdot e(n) \quad (8)$$

where, the m -th small weight vector $\mathbf{w}_m(n)$ and input vector $\mathbf{x}_m(n)$ are defined as

$$\mathbf{w}_m(n) = [w_n(mL), w_n(mL+1), \dots, w_n(mL+L-1)] \quad (9)$$

$$\mathbf{x}_m(n) = [x(n-mL), x(n-mL-1), \dots, x(n-mL-L+1)] \quad (10)$$

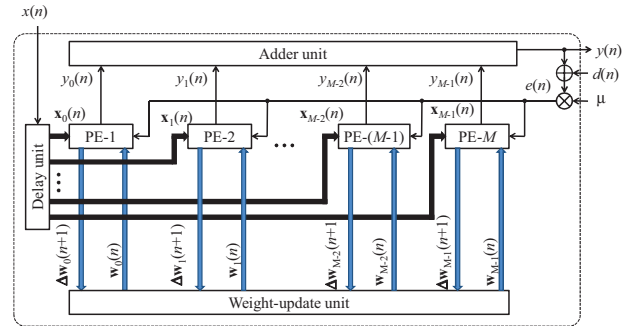


Figure 1. Proposed structure for LMS-based adaptive filter.

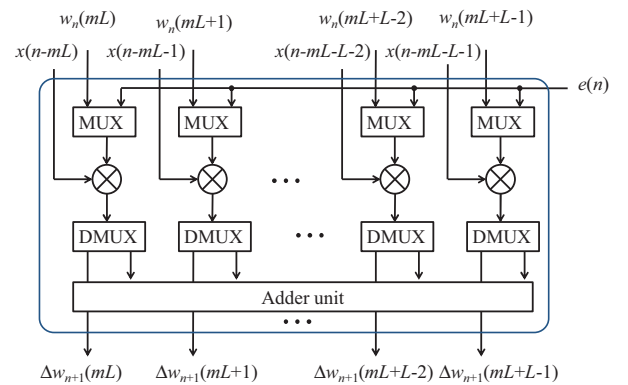


Figure 2. Internal structure of processing element.

The proposed structure for LMS adaptive filter is shown in Fig.1. It consists of M processing elements (PEs), one delay unit, one weight-update unit and adder unit. The delay unit

receives one sample of input sequence $x(n)$ in every iteration and produce one input-vector of size N using $(N-1)$ recent past samples. The input-vector split into M small vectors of size L each. The m -th PE receive small input-vector $\mathbf{x}_m(n)$, the small weight-vector $\mathbf{w}_m(n)$ and error-signal $e(n)$. It performs L -point inner-product between $\mathbf{x}_m(n)$, and $\mathbf{w}_m(n)$ to compute one partial filter output $y_m(n)$ during first half of each iteration and computes L -point vector-scalar multiplication during second half of each iteration to compute weight-increment vector $\{\Delta \mathbf{w}_m(n)\}$ for updating weight-vector $\{\mathbf{w}_m(n)\}$. The weight-update unit receives the M number weight-increment vectors from M PEs in every iteration and update the weight-vector $\{\mathbf{w}(n)\}$ for $(n+1)$ iteration. M partial filter outputs of M PEs are added in the adder unit to obtain the filter output $y(n)$. The internal structure of m -th PE is shown in Fig.2. It consists of L 2:1 multiplexers (MUXes) and L demultiplexers (DMUXes). During first half of each iteration, the m -th PE performs L -point inner product between input-vector $\mathbf{x}_m(n)$, and weight-vector $\mathbf{w}_m(n)$ to compute one partial filter output $y_m(n)$ while it performs vector-scalar product between $\mathbf{x}_m(n)$, and $e(n)$ to compute the weight-increment vector $\{\Delta \mathbf{w}_m(n)\}$. In the next section we present the FPGA implementation of the proposed architecture using Xilinx DSP48E1 Slice.

III. FPGA IMPLEMENTATION

The proposed architecture is implemented in Xilinx 7K325T FPGA. The logiCORE component DSP48 is used for implementation of multiplication and addition. The design is simulated using Xilinx system generator tool and the corresponding HDL code synthesized and implemented in FPGA using Xilinx Vivado software. The simulation was performed using Vivado simulator.

A. System Generator

The proposed simulated structure in Xilinx system generator [7] is shown in Fig. 3. The design includes Control logic unit, serial to parallel (SP) control unit and PE unit. The SP control unit multiplexes the serial incoming data to parallel and stores the parallel data to the registers. The outputs of these registers are connected to MUXs and are fed to the PE unit. The finite state machine of the control logic unit is described with Verilog and controls the entire system operations starting from filtering step to the error calculation and filter weight update step.

The PE unit consists of three Xilinx DSP48 cores and an adder tree to generate the filter output. The filter output is subtracted from desired signal to obtain the error. The error signal is scaled by the convergence factor and the result is used for scaling the input-vector to calculate the weight-increment vector for updating the current iteration weight-vector. The filtering computation is performed using parallel set of PE's to reduce the computation time. The resource sharing method is applied for implementation to reduce FPGA DSP48 usage. The number of DSP48 units used in the implementation depends on the algorithm partitions. The implemented filter has 12 taps and it is partitioned into three sections each one with 4 taps. The

Fig. 4 shows the structure of each partition. The partial product $y_m(n)$ of each partition is generated using DSP48 MAC feature and are added together to generate filter output $y(n)$ using adder-tree. The filter output $y(n)$ computation requires 6 clock cycles where 4 clock cycle consumed in MAC stage and 2 clock cycle in the adder-tree stage. The error $e(n)$ computation takes one clock.

Each PE also computes the weight-increment terms of the small weight-vectors earlier used for filter computation. The MAC unit of each PE takes 4 clock cycles to compute the weight-increment vector. All the three PEs are run in parallel and generate the weight-increment vector to update the small weight-vectors of respective PEs. Therefore, the filter computation and weight-update computation of 12-tap LMS adaptive filter is completed in 11 clock cycles which is the duration of one iteration period.

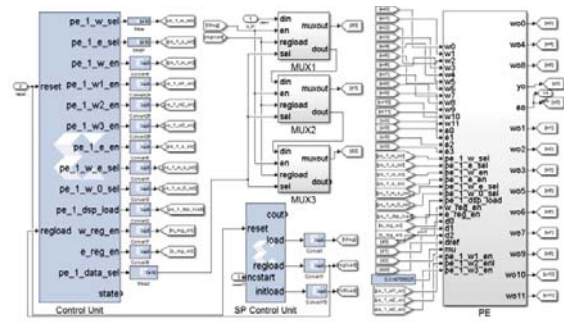


Figure 3. LMS adaptive filter in system generator.

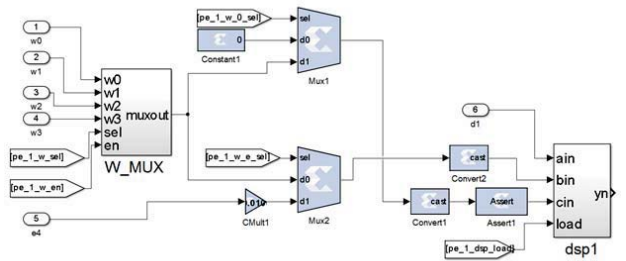


Figure 4. Detailed structure of one partition.

B. FPGA Implementation Result and Comparison

The proposed design and the existing design of [2] are modeled in system generator and both the system generator models are synthesized by Xilinx Vivado software and targeted to 7K325T Xilinx FPGA. Table 1 and Table 2 summarize the post implementation resource usage and the power consumption of the proposed architecture and those of [2]. The FPGA resource comparison shows the proposed architecture uses 3 DSP48 units compared to 36 DSP48 units of [2] for the same filter of size 12. The timing report shows that the proposed design can run with maximum 500MHz FPGA clock

frequency whereas the design for [2] run at maximum frequency 285 MHz. The proposed design support higher clock frequency is due to better utilization of DSP48 for internal pipelining addition operations. As shown in Table 2, the proposed design consumes nearly 4.7 times less dynamic power than [2]. This is mainly due to huge saving in FPGA resources.

TABLE I. FPGA RESOURCE COMPARISON.

	Proposed	Yongbo et al [2]
Resource	Utilization	Utilization
FF	1241	992
LUT	383	466
I/O	53	47
DSP48	3	36
RAMB36/FIFO	0	0
RAMB18	0	0

TABLE II. POWER ESTIMATION COMPARISON.

	Proposed	Yongbo et al [2]
Dynamic	0.051W	0.195W
Clock	0.021W	0.018W
Signals	0.010W	0.037W
Logic	0.004W	0.017W
DSP	0.015W	0.089W
I/O	0.001W	0.033W
Device Static	0.158W	0.159W

IV. SIMULATION RESULT

The LMS based adaptive filter is simulated with MATLAB Simulink. The simulation model is shown in Fig. 5. The input signal $x(n)$ is a sinusoidal signal and corrupted by random noise. The convergence factor is set to be $\mu=0.01$. The filter input $x(n)$ and filter output $y(n)$ are shown in Fig. 6 and Fig. 7 respectively. The steady state mean square error (MSE) is shown in Fig. 8.

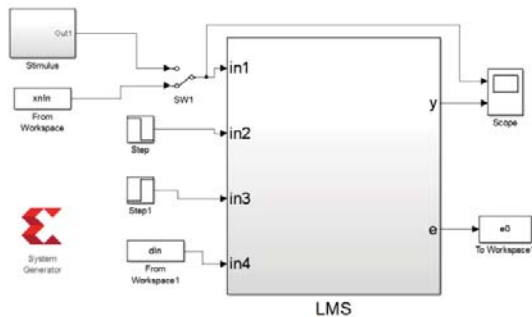


Figure 5. Simulink simulation platform.

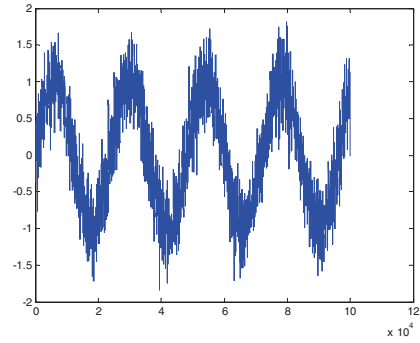


Figure 6. LMS filter input $x(n)$.

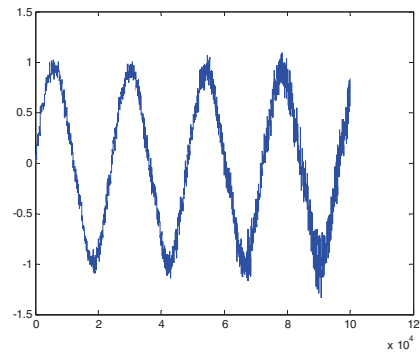


Figure 7. LMS filter output $y(n)$.

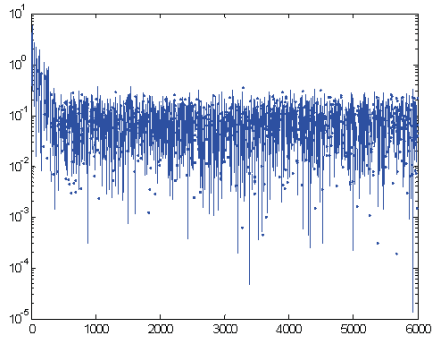


Figure 8. LMS filter MSE.

V. CONCLUSION

In this paper, we studied existing designs proposed for FPGA implementations of LMS adaptive filters and observed that excess use of multipliers and longer cycle periods are a few of the issues associated with the existing structures. We proposed a design for an FPGA implementation of an LMS based adaptive filter using the Xilinx DSP48. The proposed architecture uses one set of multipliers for the computation of both the filter output and weight-increment term. Furthermore, the filter computation and computation of the weight-increment term are decomposed and mapped to a folded structure to save FPGA multiplier resources. We have

simulated the proposed design for a 12-tap filter in Xilinx system generator and implemented the design using the Vivado tool set. The FPGA implementation results shows that the proposed architecture uses 3 DSP48 units compared to 36 DSP48 units of [2] for the same filter of size 12. Additionally, the proposed design supports a 75% higher clocking frequency than [2], and uses nearly 4.7 times less dynamic power than the architecture of [2]. These characteristics make the proposed design suitable for an efficient FPGA realization of an LMS adaptive filter for real-time digital signal processing applications.

ACKNOWLEDGMENT

We acknowledge the support from Santa Clara University PhD Pilot Research Assistant (RA) Tuition program.

REFERENCES

- [1] S. Haykin, *Adaptive Filter Theory*, 3rd ed., Upper Saddle River, New Jersey : Prentice Hall, 1996.
- [2] Yongbo Bai, Zili Chen, Ruibo Hou, Dongsheng Qi, "FPGA Implementation of A Channel Equalizer based on LMS Algorithm," 2010 First International Conference on Pervasive Computing, Signal Processing and Applications, pp 1149-1152, Sept. 2010.
- [3] Wang Yaqin; Liu Xuebin; Hu Bingliang "Implementation of a LMS filter on FPGA employing extremeDSP and smart IP-core design," 10th International Conference on Electronic Measurement & Instruments (ICEMI), vol 3, pp 341-345, 2011.
- [4] Wang Shenming; Suntiamorntut, W.; Jindapetch, N.; Ji Qiufan "Scheduling & Resources Sharing Technique for Adaptive LMS Filter," 8th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), pp 114-117, 2011.
- [5] Guanghui Cai; Jun Yang; Chuan Liang; Hongye Li, "Design and implementation of LMS adaptive filter algorithm based on FPGA," 2nd International Symposium on Instrumentation and Measurement, Sensor Network and Automation (IMSNA) and Automation (IMSNA), pp 383-385, 2013.
- [6] 7 Series DSP48E1 Slice User Guide, UG479 (v1.7), May 10, 2014.
- [7] Model-Based DSP Design using System Generator, UG879 (v2013.3) November 16, 2012.
- [8] L. D. Van and W. S. Feng, "Efficient systolic Architectures for 1-D and 2-D DLMS adaptive digital filters," in Proc. IEEE Asia Pacific Conf. on Circuits Syst., Tianjin, China, Dec. 2000, pp. 399-402.
- [9] L. D. Van, and W. S. Feng, "An efficient systolic architecture for the DLMS adaptive filter and its applications," IEEE Trans. Circuits Syst. II: Analog and Digit. Signal Process., vol. 48, no. 4, pp. 359-366, Apr. 2001.
- [10] P. K. Meher and S. Y. Park, "Critical-Path Analysis and Low-Complexity Implementation of the LMS Adaptive Algorithm," IEEE Transaction on Circuit and Systems-I: Regular Papers, vol. 61, no. 3, pp. 778-788, March 2014.
- [11] S. Y. Park and P. K. Meher, "Low-power, high-throughput, and low-area adaptive FIR filter based on distributed arithmetic," IEEE Transaction on Circuit and Systems-II: Express Briefs, vol. 60, no. 6, pp. 346-350, June 2013.
- [12] B. K. Mohanty and P. K. Meher, "A high-performance energy-efficient architecture for FIR adaptive filter based on new distributed arithmetic formulation of block LMS algorithm," IEEE Transactions on Signal Processing, vol. 61, no. 4, pp. 921-932, Feb. 15, 2013.
- [13] B.K. Mohanty and P.K. Meher, "Delayed block LMS algorithm and concurrent architecture for high-speed implementation of adaptive FIR filters," IEEE Region 10 TENCON2008 Conference, Hyderabad, Nov. 2008.